Error handling

Task 5. The API description with all error responses is in 05-openapi.yaml. It lists 12 operations. Paste it into https://editor.swagger.io to browse it. The running server also has a generated reference at `/docs`.

1. One error format

Every error, from every endpoint, has the same shape (RFC 9457 problem details):

```
{
  "type": "https://quefy.app/problems/track_already_queued",
  "title": "This track is already in the queue.",
  "status": 409,
  "code": "track_already_queued",
  "queueItemId": "0192a6f4-3c1e-7b2a-9d41-5e8f2c7a1b30",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736"
}
```

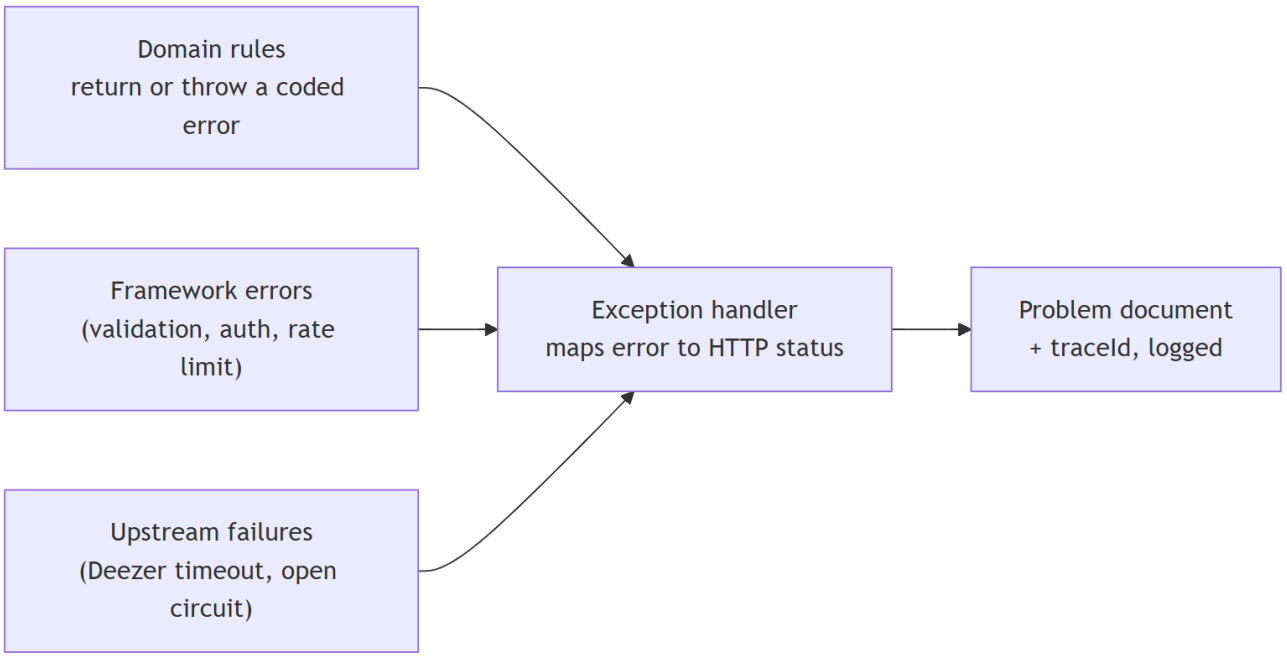
- `code` is what the app uses. It picks the translated text for the user (English or Ukrainian). `title` is for developers.
- `traceId` links the response to the server logs.
- Some codes add fields the app can use: `queueItemId` (offer "upvote instead"), `availableAt`, `limit`, `maxMinutes`, `retryAfterSeconds`, `currentRevision`, `field`.
- Server errors never contain stack traces or internal messages.

2. When each status is used

Status	Meaning	Examples of `code`
400	The request can't be read or fails model validation	`validation_failed`, `malformed_request`
401	No valid token	`unauthorized`, `refresh_token_invalid`, `refresh_token_reused`
403	Logged in, but not allowed here	`not_room_member`, `banned_from_room`, `host_only`, `owner_only`, `not_your_track`
404	It does not exist	`room_not_found`, `queue_item_not_found`, `track_not_found`, `pairing_code_invalid`, `nothing_playing`
409	Conflicts with the current state	`track_already_queued`, `track_recently_played`, `concurrency_conflict`, `room_not_active`, `idempotency_key_reused`, `invalid_state_transition`
422	Valid request, but a business rule says no	`queue_quota_exceeded`, `track_too_long`, `explicit_not_allowed`, `guests_cannot_add`, `cannot_vote_own_track`, `room_full`, `invalid_setting`

Status	Meaning	Examples of `code`
429	Too many requests, with `Retry-After`	`rate_limited`, `add_cooldown`
502 / 503	Deezer or the speaker is not available	`upstream_unavailable`, `speaker_offline`
500	Bug	`internal_error`

3. How it is built



- Business rules live in the domain and fail with a code (for example `QueueRules.EnsureCanAdd`). They know nothing about HTTP.
- One exception handler turns these into the problem document; the framework's own 400, 401, 403, 404 and 429 responses are rewritten into the same format.
- A database unique-index violation is translated to a 409 instead of leaking as a 500.
- Outgoing calls have timeouts, retries and a circuit breaker; a failure becomes 502 for the user and the circuit stays open for 15 seconds.

4. Which errors each operation can return

Operation	400	401	403	404	409	422	429	5xx
POST /auth/device	x					x	x	
POST /auth/refresh	x	x					x	
POST /rooms	x	x				x	x	
POST /rooms/by-code/{code}/join		x	x	x	x	x	x	
PATCH /rooms/{id}	x	x	x	x	x	x		
GET /catalog/search	x	x					x	502

Operation	400	401	403	404	409	422	429	5xx
POST /rooms/{id}/queue	x	x	x	x	x	x	x	
DELETE /rooms/{id}/queue/{item}		x	x	x	x			
PUT /rooms/{id}/queue/{item}/vote	x	x	x	x		x		
POST /rooms/{id}/playback/skip-vote		x	x	x	x		x	
POST /rooms/{id}/playback/{command}	x	x	x	x				503
POST /speakers/pair	x			x			x	

5. Rate limits

Limit	Value	Per
All endpoints	120 requests, refilling 40 per 10 s	user (or IP when anonymous)
Register / refresh	20 per minute	IP
Join by code	30 per minute	IP
Speaker pairing	10 per minute	IP
Search	90 per minute	user
Vibe and DJ (LLM) calls	12 per minute	user
Adding tracks	2 in a burst, then one per 20 s (host and co-host exempt)	user and room

6. Retries and duplicates

A mobile client on bad Wi-Fi will retry. Safe rules:

- GET, PUT (vote) and DELETE can be repeated without harm.
- `POST /queue` accepts an `Idempotency-Key` header. If the first request succeeded but the response was lost, the retry gets the stored response (header `Idempotent-Replayed: true`) and the track is added only once. The same key with a different body gives 409 `idempotency_key_reused`. Records are kept 24 hours.
- A 429 or 503 should be retried after `Retry-After`; 4xx errors (except 429) should not be retried.

7. Realtime and speaker errors

The WebSocket hubs report problems through events instead of status codes. A banned or removed user gets a `removed_from_room` notice and the app leaves the room. A speaker that cannot play a track sends `ReportError`; the server skips that track, tells the person who added it, and plays the next one.

8. Known gaps

- `speaker_scope_required`, `ai_budget_exceeded` and `ai_unavailable` are reserved codes that are not emitted yet. An empty AI budget silently falls back to rule-based logic.
- The extra field `status` on `invalid_state_transition` clashes with the standard `status` member.